

Security Middleware Documentation

This document describes the comprehensive security measures implemented in the Gema backend API.

Table of Contents

- 1. [Rate Limiting](#)
- 2. [Input Sanitization](#)
- 3. [Security Headers](#)
- 4. [CORS Configuration](#)
- 5. [Request Validation](#)
- 6. [Security Monitoring](#)

Rate Limiting

Overview

Rate limiting prevents abuse and brute force attacks by limiting the number of requests from a single IP address or user.

Available Rate Limiters

Limiter	Window	Max Requests	Use Case
generalLimiter	15 min	100	General API endpoints
authLimiter	15 min	5	Login, register, Firebase auth
passwordResetLimiter	1 hour	3	Password reset requests
emailVerificationLimiter	1 hour	5	Email verification
uploadLimiter	1 hour	20	File uploads
createResourceLimiter	1 hour	30	Creating events, orders
adminLimiter	15 min	200	Admin operations
searchLimiter	1 min	60	Search queries
paymentLimiter	1 hour	10	Payment processing

Implementation

```
import { authLimiter, passwordResetLimiter } from '../middleware';

// Apply to specific routes
router.post('/login', authLimiter, validateLogin, validate, login);
router.post('/forgot-password', passwordResetLimiter, validateForgotPassword, validate, forgotPassword);
```

Custom Rate Limiters

Create custom rate limiters for specific needs:

```
import { createCustomLimiter } from '../middleware';

const customLimiter = createCustomLimiter({
  windowMs: 60 * 1000, // 1 minute
  max: 10, // 10 requests per minute
  keyPrefix: 'custom',
  message: 'Too many requests to this endpoint',
});
```

Features

- **IP-based limiting:** Tracks requests per IP address
- **User-based limiting:** For authenticated routes, tracks per user ID
- **Email-based limiting:** For auth endpoints, combines IP + email
- **Skip successful requests:** Auth limiters don't count successful logins
- **Informative errors:** Clear error messages with retry-after time
- **Security logging:** Logs rate limit violations for monitoring

Input Sanitization

NoSQL Injection Prevention

The `sanitizeRequest` middleware prevents NoSQL injection attacks:

```
// Removes keys starting with $ or containing .
// Input: { $where: "malicious code" }
// Output: { _where: "malicious code" }
```

XSS Prevention

Multiple layers of XSS protection:

1. **Script tag removal:** Strips `<script>` tags from all input
2. **Event handler removal:** Removes `onclick`, `onerror`, etc.
3. **JavaScript protocol removal:** Strips `javascript:` URLs
4. **Null byte removal:** Removes null bytes from strings

Sanitization Flow

```
Request → NoSQL Sanitizer → XSS Sanitizer → Suspicious Pattern Detector → Controller
```

Selective Sanitization

Certain fields (like blog content) receive lighter sanitization:

```
// Fields with selective sanitization
const skipSanitization = ['description', 'content', 'bio', 'answer'];

// Only script tags removed, HTML formatting preserved
```

Security Headers

Helmet Configuration

Comprehensive security headers using Helmet:

```
{
  // Content Security Policy
  contentSecurityPolicy: {
    defaultSrc: ["'self'"],
    styleSrc: ["'self'", "'unsafe-inline'", 'https://fonts.googleapis.com'],
    imgSrc: ["'self'", 'data:', 'https:', 'blob:'],
    scriptSrc: ["'self'"],
    connectSrc: ["'self'", 'https://api.stripe.com'],
  },

  // HTTP Strict Transport Security
  hsts: {
    maxAge: 31536000, // 1 year
    includeSubDomains: true,
    preload: true,
  },

  // Other headers
  noSniff: true, // X-Content-Type-Options
  frameguard: { action: 'deny' }, // X-Frame-Options
  xssFilter: true, // X-XSS-Protection
}
```

Headers Added

- `Content-Security-Policy`: Prevents XSS by controlling resource sources
- `Strict-Transport-Security`: Forces HTTPS connections
- `X-Content-Type-Options`: Prevents MIME sniffing
- `X-Frame-Options`: Prevents clickjacking
- `X-XSS-Protection`: Legacy XSS protection
- `Referrer-Policy`: Controls referrer information

CORS Configuration

Allowed Origins

```
const allowedOrigins = [
  'http://localhost:3000',
  'http://localhost:5173',
  'https://gema-project.onrender.com',
  'https://kidrove-frontend.vercel.app',
  process.env.FRONTEND_URL,
];
```

Configuration

```
{
  origin: (origin, callback) => {
    // Allow requests with no origin (mobile apps)
    if (!origin) return callback(null, true);

    // Check whitelist
    if (allowedOrigins.includes(origin)) {
      callback(null, true);
    } else {
      callback(new Error('Not allowed by CORS'));
    }
  },
  credentials: true,
  methods: ['GET', 'POST', 'PUT', 'PATCH', 'DELETE', 'OPTIONS'],
  maxAge: 86400, // 24 hours
}
```

Request Validation

Size Limiting

Prevents large payload attacks:

```
// Maximum request size: 10MB
const maxSize = 10 * 1024 * 1024;

if (contentLength > maxSize) {
  return next(new AppError('Request payload too large', 413));
}
```

Suspicious Pattern Detection

Automatically detects and blocks suspicious requests:

```
const suspiciousPatterns = [
  /(\\.\\.\\.\\|\\/etc\\/|\\/proc\\/|\\/sys\\/)/i, // Path traversal
  /(union|select|insert|update|delete|drop)/i, // SQL injection
  /(<script|javascript:|onerror=)/i, // XSS
  /(\\$where|\\$ne|\\$gt)/i, // NoSQL injection
];
```

HTTP Parameter Pollution

Prevents duplicate parameter attacks:

```
// Whitelist parameters that can appear multiple times
whitelist: ['tags', 'categories', 'eventIds', 'status']
```

Security Monitoring

Request Logging

All requests are logged with:

- Timestamp
- HTTP method and URL
- Client IP address
- User agent
- Authenticated user ID (if applicable)

- Response time

Suspicious Activity Logging

The following are automatically logged:

- **Rate limit violations:** IP, email, endpoint
- **Authentication failures:** IP, attempted email
- **Blocked origins:** CORS violations
- **Suspicious patterns:** Detected attack attempts
- **Slow requests:** Requests taking >1 second
- **Server errors:** 5xx status codes

Log Examples

```
// Rate limit violation
console.warn('Rate limit exceeded for auth attempt from IP: 1.2.3.4, email: user@example.com');

// Suspicious pattern
console.error('Suspicious URL detected from IP 1.2.3.4: /api/users/../../../../etc/passwd');

// Slow request
console.warn('Slow request detected: GET /api/events - 1523ms');

// Failed auth
console.warn('Failed authentication attempt:', {
  ip: '1.2.3.4',
  email: 'attacker@example.com',
  userAgent: 'Mozilla/5.0...'
});
```

IP Blocking

Blacklist Management

Dynamically block malicious IP addresses:

```
import { blacklistIP, removeFromBlacklist } from '../middleware';

// Block an IP
blacklistIP('1.2.3.4');

// Unblock an IP
removeFromBlacklist('1.2.3.4');
```

Automatic Blocking (Future Enhancement)

Could be extended to automatically block IPs after:

- Multiple failed login attempts
- Repeated rate limit violations
- Suspicious pattern detections

Best Practices

1. Apply Rate Limiters Appropriately

```
// ☑ Good: Strict limiting for auth endpoints
router.post('/login', authLimiter, validateLogin, validate, login);

// ✗ Bad: No rate limiting on auth endpoint
router.post('/login', validateLogin, validate, login);
```

2. Use Specific Rate Limiters

```
// ☒ Good: Use specific limiters for specific operations
router.post('/upload', uploadLimiter, upload);
router.post('/reset-password', passwordResetLimiter, resetPassword);

// ☐ Bad: Using general limiter for everything
router.post('/upload', generalLimiter, upload);
```

3. Apply Security Middleware in Correct Order

```
// Correct order in app.ts
app.use(securityHeaders);
app.use(corsOptions);
app.use(requestSizeLimiter);
app.use(sanitizeRequest);
app.use(preventParameterPollution);
app.use(sanitizeInput);
app.use(detectSuspiciousActivity);
app.use(checkIPRestrictions);
app.use(securityLogger);
```

4. Monitor Security Logs

Regularly review logs for:

- Unusual patterns of failures
- High rate limit violations
- Repeated suspicious requests
- Failed authentication attempts from same IPs

5. Keep Dependencies Updated

Regularly update security packages:

```
npm update helmet express-rate-limit express-mongo-sanitize hpp cors
```

Emergency Procedures

Under Attack

If experiencing an active attack:

1. **Enable IP whitelisting** (in emergency only)
2. **Tighten rate limits** temporarily
3. **Block attacking IPs** using blacklist
4. **Review logs** to identify attack vector
5. **Apply additional protections** as needed

Rate Limit Adjustments

Temporarily increase/decrease limits during:

- **Traffic spikes:** Increase limits for legitimate users
- **Active attacks:** Decrease limits to slow attackers
- **Maintenance:** Tighten limits during maintenance windows

Testing Security Measures

Rate Limit Testing

```
# Test auth rate limiter (should block after 5 attempts)
for i in {1..10}; do curl -X POST http://localhost:5000/api/auth/login; done
```

XSS Testing

```
# Should sanitize malicious script
curl -X POST http://localhost:5000/api/test \
  -H "Content-Type: application/json" \
  -d '{"comment": "<script>alert(\"XSS\")</script>"}'
```

NoSQL Injection Testing

```
# Should sanitize $ operators
curl -X POST http://localhost:5000/api/test \
  -H "Content-Type: application/json" \
  -d '{"email": {"$ne": null}}'
```

Security Checklist

- ☒ Rate limiting on all public endpoints
- ☒ Strict rate limiting on authentication
- ☒ NoSQL injection prevention
- ☒ XSS prevention and input sanitization
- ☒ Security headers (Helmet)
- ☒ CORS configuration
- ☒ Request size limiting
- ☒ Suspicious pattern detection
- ☒ HTTP parameter pollution prevention
- ☒ Security logging and monitoring
- ☒ IP blacklist functionality
- ☐ Automated IP blocking (future)
- ☐ Intrusion detection system (future)
- ☐ Web Application Firewall (future)

Related Documentation

- [Rate Limiting \(./rateLimiter.ts\)](#)
- [Security Middleware \(./security.ts\)](#)
- [Validation \(./validation.ts\)](#)
- [Authentication \(./auth.ts\)](#)